



ContractSentry: a static analysis tool for smart contract vulnerability detection

Shiji Wang¹ · Xiangfu Zhao¹

Received: 2 April 2024 / Accepted: 3 October 2024 / Published online: 23 October 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Frequent smart contract security incidents pose a threat to the credibility of the Ethereum platform, making smart contract vulnerability detection a focal point of concern. Previous research has proposed vulnerability detection methods in smart contracts. Generally, these tools rely on predefined rules to detect vulnerable smart contracts. However, using out-of-date rules for vulnerability detection may lead to a significant number of false negatives and false positives due to the growing variety of smart contract vulnerability types and the ongoing enhancement of vulnerability defense mechanisms. In this paper, we propose ContractSentry, a tool for static analysis of smart contracts. First, we preprocess Solidity code to build critical contract information and transform it into an intermediate representation. Then, based on the intermediate representations, we propose composite rules for vulnerability detection by analyzing the characteristics of different types of vulnerabilities in smart contracts. Finally, we evaluate ContractSentry with two datasets and compare it with state-of-the-art vulnerability detection tools. Experimental results demonstrate that ContractSentry achieves superior detection effectiveness.

Keywords Ethereum · Smart contracts · Static analysis · Vulnerability detection · Intermediate representation · Composite rules

1 Introduction

Blockchain technology (Guo and Yu 2022) has achieved rapid development and wide application since its introduction in 2008. As a decentralized, tamper-proof, and irreversible distributed ledger, blockchain has advanced numerous industries in addition

✉ Xiangfu Zhao
xiangfuzhao@gmail.com

Shiji Wang
shijiwang00@163.com

¹ School of Computer and Control Engineering, Yantai University, 30 Qingquan Road, Yantai 264005, Shandong, China

to its success in the cryptocurrency field. Bitcoin (Nerurkar et al. (2021) as the first cryptocurrency to use blockchain technology, aims to execute transactions and store currency but has limited functionality that cannot fully leverage the characteristics of blockchain technology. On the other hand, Ethereum (Wood et al. 2014) as a distributed blockchain platform, takes full advantage of blockchain technology and provides a programmable environment that allows developers to build and execute smart contracts (Chen et al. 2023).

As one of the most successful applications of blockchain technology, smart contracts have attracted widespread interest in both research and industries. Their applications have expanded to various fields, including logistics (Alqarni et al. 2023), finance (John et al. 2023), Internet of Things (Wang et al. 2023), and gaming (Boroń and Kobusińska 2021), etc. Solidity (2023a) is the most popular language for writing smart contracts due to its mature tool support and Turing completeness. The majority of smart contracts running on the Ethereum platform are written in Solidity. A smart contract cannot be altered once it has been successfully deployed on the Ethereum platform. If not properly written, it may lead to potential vulnerabilities in smart contracts being exploited by attackers, causing irreversible damage. Reentrancy is one of the most notorious vulnerabilities in smart contracts. In the DAO attack (2023b) in 2016, hackers exploited this vulnerability successfully stole over 60 million dollars worth of ether, and led to a hard fork of the Ethereum main chain. The Parity wallet (2023c) was found to have a serious vulnerability in 2017. This vulnerability allowed hackers to freeze and control user funds by exploiting a vulnerability in the contract. It resulted in a significant amount of ether and other tokens being controlled by the hackers. Security vulnerabilities in smart contracts are a persistent and exploitable problem rather than individual incidents. According to data from SlowMist Hacked (2023), economic losses due to Ethereum smart contract vulnerabilities have exceeded 3 billion dollars. In 2023 only, the total losses amounted to over 400 million dollars.

Witnessing this situation, many researchers have proposed various tools for detecting vulnerabilities in smart contracts. However, a recent study (Durieux et al. 2020) showed that existing tools perform poorly in detecting real-world smart contract vulnerabilities, with significant levels of false positives and false negatives. Oyente (Luu et al. 2016) analyzes smart contracts using symbolic execution techniques, exploring potential execution paths and effectively identifying potential vulnerabilities. Yet, when handling a huge contract with many *jump* instructions, it could experience problems like state space explosion, and the assertion mechanisms/rules used to verify vulnerabilities are not comprehensive enough. ContractFuzzer (Jiang et al. 2018) is a smart contract vulnerability detection tool based on fuzz testing. Due to the limited program execution states covered by the test cases, it is unable to achieve the desired path coverage. Furthermore, the incomplete test oracles/rules used to detect vulnerabilities lead to unsatisfactory results.

In order to find vulnerabilities in smart contracts, researchers have also explored the possibility of transforming the source code or bytecode into an intermediate representation with high semantic expressiveness and then analyzing the intermediate representations using predefined rules. Slither (Feist et al. 2019) and Securify (Tsankov et al. 2018) are both detection tools that verify contract security by constructing an intermediate representation and applying predefined rules. However, the effectiveness

of these tools is significantly reduced due to their outdated logical rules. On the one hand, existing vulnerability detection rules might not match some vulnerable code in smart contracts. On the other hand, the emergence of new defenses has effectively stopped vulnerabilities from arising at the root, but contracts may still match these predefined patterns. To prevent such circumstances, it is necessary to learn more comprehensive rules.

Based on the above insights, we propose an automated static analysis tool called ContractSentry for detecting smart contracts on Ethereum. The intermediate representations of the tool are constructed at the contract source code level. And before building the intermediate representations, we extract crucial contract information such as the control flow graph. Vulnerability detection is based on the analysis of the intermediate representations. ContractSentry combines defense mechanisms and defines comprehensive detection rules for identifying three types of vulnerabilities: *Reentrancy*, *Denial of Service*, and *Transaction Ordering Dependence*. According to evaluation results, ContractSentry reduces false positives and false negatives more effectively than many other state-of-the-art tools (i.e., Slither, Oyente, Securify, Sailfish (Bose et al. 2022), and Smartian (Choi et al. 2021)).

In summary, this paper makes the following contributions:

- We propose new composite rules. Based on the intermediate representation, we propose composite rules to distinguish between benign vulnerability contracts and truly vulnerable contracts and ensure that these rules are universal.
- We propose seven detection algorithms for different vulnerabilities and defense mechanisms to apply composite rules effectively.
- We evaluate ContractSentry on two datasets and show that it outperforms all five existing approaches used as a baseline, in terms of Precision, Recall, and F1.

The rest of the paper is organized as follows. Section 2 introduces the Solidity language and the rationale for the vulnerabilities to be addressed, as well as our motivation for this work. Section 3 describes the basic steps of ContractSentry, the tool we propose for detecting smart contract vulnerabilities. Section 4 provides an introduction to the intermediate representations of ContractSentry. Section 5 details how our tool conducts vulnerability detection through rules. Section 6 compares ContractSentry with many other state-of-the-art tools by testing two datasets. Section 7 discusses related work in the field. Finally, Sect. 8 concludes our work.

2 Background and motivation

In this section, we briefly introduce the Solidity language and the three types of vulnerabilities (Khan and Namin 2020): *Reentrancy*, *Denial of Service*, and *Transaction Ordering Dependence*, and explain the motivation for our research.

2.1 Solidity

Solidity is a programming language applied to smart contracts. It is created for the Ethereum platform and interacts with the Ethereum Virtual Machine (EVM). Smart

contracts are computer programs that automatically execute contract conditions. They are stored on the blockchain and can be executed without the third-party intervention. Solidity is similar to JavaScript and has features such as static typing and object orientation. It provides a rich set of functionalities such as functions, events, state variables, and structures to support the writing of smart contracts. With Solidity, developers can define the logic of the contract and realize operations such as contract state changes.

2.2 Vulnerability types

2.2.1 Reentrancy

Reentrancy (Zheng et al. 2023) may arise when a smart contract interacts with other smart contracts through external calls. The attacker uses the mechanism of an external call to recursively call the compromised contract repeatedly and obtain additional funds on each call.

In Listing 1, the *withdrawFunds()* allows users to withdraw a certain amount of funds from their balances. First, it verifies that the account has enough balance to pay for the requested amount (line 4). Then, it sends ether to the caller (line 5) and finally deducts the withdrawn amount from the balance of the caller (line 6). However, this seemingly normal behavior is vulnerable to reentrancy. When the function executes to line 5, it triggers the fallback() of the attacker and the attacker can call the *withdrawFunds()* of the target contract again. Since the balance update operation (line 6) comes after the transfer operation is completed (line 5), each recursive call to the function still satisfies the condition in line 4. Since the balance update operation (line 6) is after the transfer operation is completed (line 5), the condition in line 4 is still satisfied each time the function is reentered. The attacker can repeatedly call the function until the balance of the target contract is exhausted or the condition set by the attacker is met.

Listing 1 An example of Reentrancy vulnerability

```
1  contract EtherStore {
2    ...
3    function withdrawFunds () public {
4        require(balances[msg.sender] >= _weiFunds);
5        msg.sender.call.value(_weiFunds)();
6        balances[msg.sender] -= _weiFunds;
7    }
8 }
```

2.2.2 Denial of Service(DoS)

Denial of Service (Chen et al. 2020) is one of the most common vulnerabilities in smart contracts, arising mostly because the change in the state of a smart contract depends

on the execution result of an external call. An attacker can prevent the contract from executing or providing normal services by destroying the original logic in the contract.

Listing 2 shows a contract for throne bidding. If a bidder bids higher than the current bid (i.e., *highestBid*) for the throne, the current bid will be returned to the current throne owner (line 7). Subsequently, the throne will be handed over to the bidder and the bid will be updated (lines 8-9). However, returning the bid to the throne owner will trigger the *fallback()* of the owner. If the throne owner is a malicious user with an exception thrown in the *fallback()*, the transaction will be rejected and rolled back. No matter how high the bid amount the bidder provides, the same situation will occur. Eventually, no one will be able to bid successfully.

Listing 2 An example of DoS vulnerability

```

1  contract KotET{
2      address currentking;
3      uint highestBid;
4      ...
5      function bid() public payable{
6          require(msg.value > highestBid);
7          if(!currentking.send(highestBid))revert();
8          currentking = msg.sender;
9          highestBid = msg.value;
10     }
11 }
```

2.2.3 Transaction Ordering Dependence(TOD)

The Ethereum network processes transactions through blocks, each containing transactions over a while. The execution order of the transactions is determined by miners, and miners prioritize transactions with higher gas fees. If an attacker exploits this feature and monitors the transaction of the attacked contract, he/she pays a higher gas fee to send his/her transaction, causing both his/her transaction and the transaction of the attacked contract to be packaged in the same block. The miner gives priority to processing the transaction of the attacker, resulting in losses to the attacked contract and the TOD vulnerability arises (Qian et al. 2022).

For example, Listing 3 shows a partial fragment of a smart contract with two functions, *RewardStandard()* and *ClaimReward()*. *RewardStandard()* is used for the contract owner to set the reward amount and *ClaimReward()* is used for claiming the reward. However, the contract owner can continuously monitor the blockchain network and if a user sends a transaction to claim the reward, the contract owner sends a transaction with a higher gas fee to modify the reward amount. The miner processes the transaction with the higher gas fee first, causing the reward amount received by the user to change.

Listing 3 An example of TOD vulnerability

```

1  function RewardStandard() public payable {
2      if(msg.sender == owner){
3          if(locked)revert();
4          owner.transfer(reward);
5          reward = msg.value;
6      }
7  }
8  function ClaimReward() public payable {
9      if(locked)revert();
10     if(sha256(msg.data) < difficult){
11         msg.sender.transfer(reward);
12         locked = true;
13     }
14 }

```

2.3 Motivation

Previous studies (Luu et al. 2016; Jiang et al. 2018; Feist et al. 2019; Tsankov et al. 2018) have done extensive work on vulnerability detection by customizing rules to detect vulnerabilities in smart contracts. These rules are designed to detect certain vulnerabilities, such as simple reentrancy vulnerabilities. Many tools consider that two operations *read* and *write* to state variables following an external call may induce reentrancy. However, the setting of such a pattern is insufficient to determine whether a contract truly has a reentrancy vulnerability, which may result in false positives. The function shown in Listing 4 appears to have a reentrancy vulnerability (lines 3–5), however, the *withdraw()* function is protected by identity control, which checks the identity of the caller before an external call is made (line 2) to prevent reentrancy from occurring. Slither, Oyente, and Securify mark the example in Listing 4 as vulnerable, resulting in a false alarm.

2.3.1 Our solution

In contrast, ContractSentry distinguishes between benign reentrancy and vulnerable reentrancy. Our strategy is to first eliminate functions that are not vulnerable to reentrancy, and then analyze the relationships between information within vulnerable functions. For example, for the function in Listing 4 we first find the external call that exists in the function (line 4), and then find out whether there are any restrictions on the caller (*msg.sender*) before the external call. In this way ContractSentry determines that this function is benign reentrancy, indicating that there is no reentrancy risk.

Listing 4 An example of non-vulnerable code

```
1  function withdraw(uint amount) external {  
2      require(msg.sender == owner);  
3      require (credit[msg.sender]>= amount);  
4      require( msg.sender.call.value(amount)());  
5      credit[msg.sender]-=amount;  
6  }
```

3 Overview

3.1 Static analysis method

Static analysis methods for smart contract vulnerabilities do not require executing the contract but rather detect vulnerable behavior patterns through methods such as pattern matching or semantic analysis. Static analysis tools for smart contracts are mainly divided into analyzing smart contract vulnerabilities from the EVM bytecode level and the Solidity source code level.

3.1.1 EVM bytecode analysis

EVM bytecode is the set of underlying instructions for Ethereum smart contracts and is used to execute contract code on the EVM. The actual behavior and execution path of a contract can be directly examined by analyzing it at the bytecode level, which can indeed detect some security issues. However, the lack of certain semantic details in bytecode may lead to inaccuracy and incompleteness of the analysis, resulting in false positives and false negatives.

3.1.2 Solidity source code analysis

Analyzing smart contracts using the Solidity source code allows for the preservation of rich semantic information and context. Source code provides more structural information that can help static analysis tools to better understand and reason about the behavior of contracts to identify potential vulnerabilities more accurately. Therefore, we choose to analyze smart contracts at the source code level.

3.2 ContractSentry overview

ContractSentry is a static analysis tool for smart contract detection. As shown in Fig. 1, the detection process is divided into four modules. ContractSentry takes the contract source code as input, compiles it into the abstract syntax tree (AST), and generates a detection report as output.

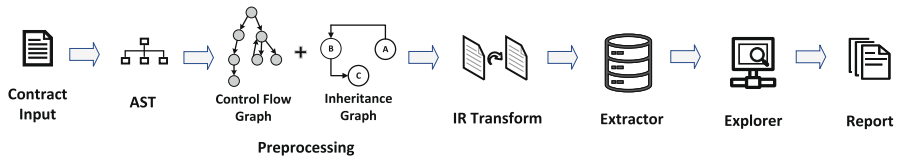


Fig. 1 The framework of ContractSentry

3.2.1 Preprocessing

The purpose of preprocessing is to build the critical information of the contract, including the control flow graph and inheritance graph. The control flow graph analyzes the contract execution flow and possible paths and deduces the potential behaviors on different execution paths. The inheritance graph analyzes the functions, variables, modifiers, and so on inherited from the parent class. The purpose of the preprocessing module is to provide critical information to the Extractor and Explorer.

3.2.2 IR transform

The second module transforms the smart contract source code into an intermediate representation (IR) to achieve high-precision analysis. Detailed explanations are provided in Sect. 4.

3.2.3 Extractor

The crucial information from the contract is identified in the Extractor phase and applied to the Explorer. For example, variables written are extracted and filtered according to their types (e.g., whether they are state variables) from every contract, function, or node in the control flow graph. Another example is capturing data dependency between variables, by passing in the function context arguments, we can identify variable dependencies within each function.

3.2.4 Explorer

First, based on the extracted information and analysis IR, Explorer makes vulnerability detection rules and generates detection patterns for different types of vulnerabilities. Then, the contract is pattern-matched and outputs whether it has some kind of vulnerability or not. Section 5 describes the details of this module.

4 IR transform

In this section, we explain why we transform Solidity code into an intermediate representation and how we do it.

4.1 Transformation motivation

The purpose of transforming the Solidity code into an intermediate representation in ContractSentry is to standardize some special cases of the Solidity language to improve the effectiveness and accuracy of the analysis. Solidity is a special language with some peculiarities, both in terms of syntax and semantics. By transforming code into an intermediate representation, these peculiarities can be eliminated and provide a more uniform basis for vulnerability analysis. For example, in Solidity, there are two types of function calls: internal calls and external calls. Internal calls refer to calls between functions within the same contract, while external calls refer to calls between functions of different contracts. Smart contract vulnerabilities are often caused by attackers exploiting external calls to contracts. External calls are further categorized into various cases, each of which has subtle differences in each case. Therefore, ContractSentry performs further categorization of external calls after transforming the Solidity code into an intermediate representation, allowing for a more accurate analysis of the impact of external calls on program security.

In addition, the intermediate representations operation instructions have a hierarchical structure that allows for a deeper analysis of potential vulnerabilities. In Listing 5, the *claim()* calls *TransferHelper.safeTransferETH()* to implement the transfer of ether from the contract address to the user address (line 2). To determine if the function is truly vulnerable, it is necessary to perform a deep analysis of its argument list after transforming it to an intermediate representation. This approach offers numerous static guarantees. By further analyzing the IR, ContractSentry can detect the presence of common vulnerabilities and determine if they can be reached under specific conditions.

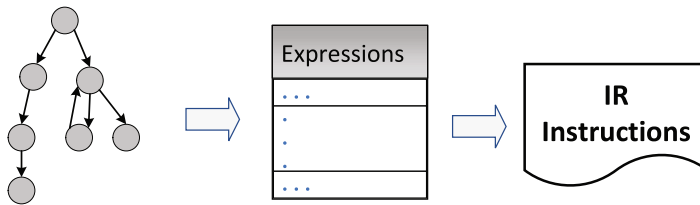
Listing 5 An example of a function

```

1  function claim() public {
2      TransferHelper.safeTransferETH(msg.sender,
3          balances[msg.sender]);
4      balances[msg.sender] = 0;
5  }
```

4.2 Process

In our study, SlithIR (2023d) instructions are extracted to meet the detection of specific vulnerability types in ContractSentry. This approach filters out irrelevant instructions and provides essential ones for IR detection, enhancing detection accuracy and efficiency. The process of transformation is shown in Fig. 2. After transforming the Solidity code into a control flow graph, each node of the control flow graph represents a basic block that contains one or more statements. Expressions and control flow graph nodes are connected through control edges, which represent control flow transfers in the code, connecting one node to another. During the transformation process, the iden-



Control Flow Graph

Fig. 2 The process of IR transformation

tified expressions are transformed into corresponding IR instructions. Each node of the control flow graph contains at most one Solidity expression that is transformed into a set of IR instructions. This representation not only simplifies the analysis process but also preserves important semantic information in the Solidity source code, which provides convenience for subsequent vulnerability analysis operations.

Listing 6 shows an example of transforming the function in Listing 5 into IR. The first expression in line 3 calls *TransferHelper.safeTransferETH()* to transfer the balance to *msg.sender*. In the IR instructions, *REF_1(uint256)* represents dereferencing the value of *balances[msg.sender]* and processes it as a value of type *uint256* (line 5). The *LIBRARY_CALL* instruction in line 6 indicates the type of call and is a kind of *HIGH_LEVEL_CALL* (that is, a user-defined function call), with other call types including *LOW_LEVEL_CALL* (e.g., *call.value()*), *SEND* (transfer using *send()*), and *TRANSFER* (transfer()). *dest:TransferHelper* indicates that the target object when executing the call instruction is *TransferHelper*. *arguments:['msg.sender', 'REF_1']* indicates the list of arguments in the function call (line 8). The second expression *balances[msg.sender] = 0* assigns 0 to *balances[msg.sender]*, i.e., the balance of *msg.sender* is set to 0 (line 9). In the IR instructions, *REF_2(uint256)* is used to dereference the value of *balances[msg.sender]*, and the *REF_2 (->balances):= 0(uint256)* instruction assigns it to 0 (lines 11-12).

Listing 6 An example of IR transformation

```

1  Contract Balance
2  Function claim
3    Expression: TransferHelper.safeTransferETH(msg.sender, balances[
      msg.sender])
4    IRs:
5      REF_1(uint256) → balances[msg.sender]
6      LIBRARY_CALL, dest:TransferHelper,
7      function:TransferHelper.safeTransferETH(address, uint256),
8      arguments:['msg.sender', 'REF_1']
9      Expression: balances[msg.sender] = 0;
10   IRs:
11     REF_2(uint256) → balances[msg.sender]
12     REF_2 (→ balances) := 0(uint256)

```

5 Explorer

In this section, we first describe the rules we propose and explain their meaning, followed by how Explorer applies these rules for vulnerability detection analysis.

5.1 Rules building

Explorer generates rules through extracted critical information and generated intermediate representations. Table 1 presents basic rules along with their explanations.

To be used for the different vulnerability detection patterns in Sect. 5.2, shown in Table 2, we propose some composite rules based on the rules in Table 1. We explain the meaning of them one by one as follows. (Note: in Table 2, $\vee$ means union and $\succ$ denotes the execution order)

Call: The external call is a low-level call or a user-defined function call.

AllCall: The external call can be of all four types in Table 1.

ConCall: Locate the node of a conditional statement (`is_conditional(n)`), and the node contains *AllCall*.

modifier_reenlock: In the modifier function, locate the node containing a conditional statement (`is_conditional(n)`) and where the variable (`var`) within the node is assigned twice (`is_dependent(n.var,_)`).

internal_reenlock: Within the function, first, locate a node that contains a conditional statement (`is_conditional(n)`). Next, there is an assignment operation on the variable within the node (`is_dependent(n.var,_)`). After an external call, the variable is assigned again (`is_dependent(n.var,_)`).

write_prior_call(var): The variable write (`var_written`) is performed first, followed by the external call operation (*AllCall*).

Table 1 Rules and explanation

Rules	Explanation
ir	Perform ir analysis on nodes
is_conditional(n)	Node n contains conditional statements (e.g., require, assert, etc.)
is_instance(x, y)	x is a type of y
Send	send()
Transfer	Transfer()
LowLevelCall	Low-level calls (e.g. call.value())
HighLevelCall	User-defined function calls
is_dependent(x,y)	x (variable or list of variables) has a data dependency on y (e.g. variable assignment operations)
for_loop(n)	A loop statement exists for node n
var_written	Variables written
SolidityVariable	Solidity language-specific variables (e.g. msg.sender)

Table 2 Composite rules

Composite Rules	Composition
Call	LowLevelCall $\vee$ HighLevelCall
AllCall	LowLevelCall $\vee$ Send $\vee$ Transfer $\vee$ HighLevelCall
ConCall	is_conditional(n) $\succ$ is_instance(ir, AllCall)
modifier_reenlock	is_conditional(n) $\succ$ is_dependent(n.var,_) $\succ$ is_dependent(n.var,_)
internal_reenlock	is_conditional(n) $\succ$ is_dependent(n.var,_) $\succ$ is_instance(ir, Call) $\succ$ is_dependent(n.var,_)
write_prior_call(var)	var_written $\succ$ is_instance(ir, AllCall)
write_after_call(var)	is_instance(ir, AllCall) $\succ$ var_written

write_after_call(var): The external call operation (AllCall) is performed first, and then the variable is written (var_written).

Note: Vulnerabilities and vulnerability defense mechanisms are our main concerns, and we design composite rules based on the basic principles of both to ensure universality. Even if the vulnerabilities or vulnerability defense mechanisms in the contract are updated, as long as their basic principles are satisfied, they can be effectively identified by composite rules.

5.2 Vulnerability detection methods

5.2.1 Reentrancy

With the development of smart contracts, some new defense mechanisms have emerged to effectively avoid reentrancy vulnerabilities. However, previous research on pre-defined rules to detect it has resulted in a large number of false positives and false

negatives. Therefore, in the Explorer module, we propose brand new rules to distinguish between benign reentrancy (i.e., *Identity Control*, *Reentrancy Lock*, *Fixed Address*, and *No Financial Risk*) and vulnerable reentrancy, aiming to reduce the generation of false positives.

Identity control. Listing 7 displays a code snippet of a real-world smart contract where reentrancy is detected in the function *call.value()* in line 8. However, due to the *onlyOwner* modifier in line 1, this function can only be called by the contract owner and not by arbitrary addresses. This restriction ensures that only a specific smart contract has the right to call the function, thus effectively preventing other malicious contracts from performing reentrancy attacks on the function.

Listing 7 Code example: Identify Control

```

1  modifier onlyOwner {
2      require(msg.sender == owner);
3      _;
4  }
5  ...
6  function enforce(uint amount) external onlyOwner{
7      ...
8      require(msg.sender.call.value(amount)());
9      ...
10 }
```

To detect the *Identity Control* to prevent false positives, two requirements must be met:

- (1) There exists an external call (*Call*), which refers to both a low-level call (such as *call.value()*) or a high-level call (user-defined calls). Two functions *send()* and *transfer()* are not considered here because they only pass a maximum of 2,300 gas to execute the *fallback()*, preventing malicious recursive calls (i.e., reentrancy) from occurring.
- (2) There are constraints within the function modifier or within the function that are specific to the caller. For instance, line 2 in Listing 7 constrains the identity of *msg.sender* to be *owner*.

Algorithm 1 describes the method for detecting *Identity Control*. Firstly, if a function contains an external call, the algorithm iterates its modifier and internal nodes (lines 5-6). If node *n* is a conditional node and directly uses a *Solidity Variable* that is not an empty address within the node (line 7), add the variable to *conditional_vars* (line 8). Lastly, if the caller is in *conditional_vars* (line 9), output the presence of *Identity Control*.

Reentrancy lock A *Reentrancy Lock* can be categorized into a modifier reentrancy lock and using a reentrancy lock inside a function. Listing 8 provides an example of using a modifier reentrancy lock to prevent reentrancy, where based on the external call in line 13, it can be detected that the *withdraw()* in line 11 has a reentrancy vulnerability.

Algorithm 1 Detection of Identity Control**Input:** a smart contract**Output:** there exists Identity Control or not

```

1: funcs  $\leftarrow$  get Call functions list;
2: nodes  $\leftarrow$  all nodes in a function/modifier;
3: conditional_vars  $\leftarrow$  set of Solidity variables directly used in conditions;
4: caller  $\leftarrow$  the contract caller;
5: for f in funcs do
6:   for n in f.nodes and f.modifier.nodes do
7:     if is_conditional(n) and is_instance(var, SolidityVariable) and not is_empty(var.expression)
       then
8:       conditional_vars  $\leftarrow$  var;
9:       if caller in conditional_vars and then
10:        return Identity Control;
11:       end if
12:     end if
13:   end for
14: end for

```

However, the modifier *nonReentrant* ensures the security of this function. This modifier is declared in line 4 and checks the state of the *notEntered* variable before executing the function and sets it to false (lines 5-6). The "_" in line 7 is a placeholder for the function body, and line 8 is only executed after the function execution is completed. If an external contract attempts to reenter the *withdraw()* before the first execution is completed, the check operation in line 5 will fail, and the transaction will be stopped. This method ensures the security of the contract.

In Listing 9, using a reentrancy lock inside a function ensures the security of the contract. It checks the status of the *mutex* variable (lines 2-3) before the external call in line 5, and the *mutex* variable state is changed only after the call completes (line 8).

Listing 8 Code example: modifier reentrancy lock

```

1  contract BANK_SAFE {
2      bool private _notEntered = true ;
3      ...
4      modifier nonReentrant () {
5          require(_notEntered);
6          _notEntered = false ;
7          _;
8          _notEntered = true ;
9      }
10     ...
11     function withdraw() external nonReentrant {
12         ...
13         require(msg.sender.call.value(amount)());
14         ...
15     }
16 }

```

Listing 9 Code example: using a reentrancy lock inside a function directly

```

1  function withdraw(...) external {
2      if (mutex == false) {
3          mutex = true;
4          ...
5          IERC20(token).transfer(recipient, amount);
6          ...
7      }
8      mutex = false;
9  }

```

Reentrancy Lock detection requires the satisfaction of the following two conditions:

- (1) There exists a *Call*.
- (2) The *Reentrancy Lock* is used in a function modifier or within a function.

Algorithm 2 is designed to detect the Reentrancy Lock. First, it iterates through the modifier and the function internal nodes of the function containing the external call operation (lines 3-4), then it determines whether a reentrancy lock is present in the modifier or using a reentrancy lock inside a function directly (line 5), and finally it outputs the results (line 6).

Fixed address. The operation of transferring funds to a fixed address does not involve reentrancy risk. An external call exists in line 6 of the *enroll()* in Listing 10. From the address *developer*, the function makes a call to the *transferFrom()* in the contract.

Algorithm 2 Detection of Reentrancy Lock**Input:** a smart contract**Output:** there exists Reentrancy Lock or not

```

1: funcs ← get Call functions list;
2: nodes ← all nodes in a function/modifier;
3: for f in funcs do
4:   for n in f.nodes and f.modifier.nodes do
5:     if f meets modifier_reenlock or internal_reenlock then
6:       return Reentrancy Lock;
7:     end if
8:   end for
9: end for

```

However, we can see that the address *developer* is assigned a hard address in line 2 that cannot be modified. Therefore, this contract is not easily exploitable and does not pose a significant risk of loss.

Listing 10 Code example: Fixed Address

```

1  contract Crowdsale {
2    ERC20 private developer = ERC20(0
3      x6B175474E89094C44Da98b954EedeAC495271d0F);
4    function enroll(...) public {
5      ...
6      developer.transferFrom(msg.sender, vault, payment);
7      ...
8    }
9  }

```

The requirements for detecting *Fixed address* must be met:

- (1) There exists a *Call*.
- (2) The destination address of the external call is assigned a hard address.

Algorithm 3 shows how to detect *Fixed Address*. Firstly, it iterates through functions that contain *Call* operations. By traversing the IR transformation instructions of each node, it identifies external call nodes (lines 5-8). The target address in the IR instruction of this node is further analyzed, and if the target address satisfies the address type and its destination address expression meets *is_address* (which means that the target address is not null or not a 0 address), the algorithm outputs a result indicating that the external call function within the contract is safe (lines 9-10).

No financial risk. Line 3 in Listing 11 defines an external call to the address *config*, and reentering this function does not pose a financial risk. The *convertToken()* in line 3 allows the first argument *msg.sender* to send tokens to the second argument (*this* represents the smart contract), so the *convertToken()* transfers tokens from the function caller to this smart contract. The generation of reentrancy vulnerability only raises the balance of this smart contract, so there is no financial risk to the smart contract.

Algorithm 3 Detection of Fixed Address**Input:** a smart contract**Output:** there exists Fixed Address or not

```

1: funcs ← get Call functions list;
2: nodes ← all nodes in a function;
3: irs ← all IR instructions in a node;
4: dest ← destination address;
5: for f in funcs do
6:   for n in f.nodes do
7:     for ir in n.irs do
8:       if is_instance(ir, Call) then
9:         if is_instance(ir.dest.type, address) and is_address(ir.dest.expression) then
10:          return Fixed Address;
11:        end if
12:      end if
13:    end for
14:  end for
15: end for

```

In Listing 12, although there is an external call (line 4), the argument for the transfer amount is *msg.value*, meaning that the ether transferred to the external address is the ether received by the *MSS()*. Reentering this function has no effect on the balance of this smart contract.

Listing 11 Code example: No Financial Risk (a)

```

1  function Platform(...) public{
2    ...
3    IConfig(config).convertToken(msg.sender,
4      address(this), amount);
5    ...
6  }

```

Listing 12 Code example: No Financial Risk (b)

```

1  function MSS(...) public payable {
2    ...
3    require(msg.value == stakeAmount);
4    require(_to.call.value(msg.value)());
5    ...
6  }

```

For the detection of *No Financial Risk*, the following conditions must be met:

- (1) There exists a *Call*.
- (2) The transfer recipient called is this smart contract or the argument of the transfer amount is *msg.value*.

Algorithm 4 shows the detection process of *No Financial Risk*. For the function that meets the existence of an external call, its internal nodes are traversed (lines 6–8), and if the IR instruction of the detection node has a *Call* (lines 8–9), its arguments are analyzed in depth. If the argument of the external call depends on *this* or the amount of argument depends on the *msg.value* (line 10), then output the result.

Algorithm 4 Detection of No Financial Risk

Input: a smart contract

Output: there exists No Financial Risk or not

```

1: funcs ← get Call functions list;
2: nodes ← all nodes in a function;
3: irs ← all IR instructions in a node;
4: args ← external call arguments in an ir;
5: call_value ← parameter of amount;
6: for f in funcs do
7:   for n in f.nodes do
8:     for ir in n.irs do
9:       if is_instance(ir, Call) then
10:        if is_dependent(args, "this") or is_dependent(call_value, "msg.value") then
11:          return No Financial Risk;
12:        end if
13:      end if
14:    end for
15:  end for
16: end for

```

5.2.2 Vulnerable reentrancy detection

Three conditions need to be met for *Reentrancy* detection:

- (1) An external call (*Call*) is present.

- (2) There is an external call to a function that does not have any of the above four conditions (i.e., *Identity Control*, *Reentrancy Lock*, *Fixed Address*, and *No Financial Risk*).
- (3) After the external call, the state variable is altered.

Algorithm 5 provides the detection rules for *Reentrancy*. For a function that contains an external call operation, the *Call* is found by traversing the IR instructions of the node within the function (lines 5-8). If there is *no_safety_report* (that is, none of the four cases above to prevent false positives) and it is not *read_prior_call(svs)* (a state variable is written before an external call), a reentrancy vulnerability report is generated (lines 9-11).

Algorithm 5 Detection of Reentrancy

Input: a smart contract

Output: there exists Reentrancy or not

```

1: funcs ← get Call functions list;
2: nodes ← all nodes in a function;
3: irs ← all IR instructions in a node;
4: svs ← state variables;
5: for f in funcs do
6:   for n in f.nodes do
7:     for ir in n.irs do
8:       if is_instance(ir, Call) then
9:         if no_safety_report() then
10:           if not write_prior_call(svs) then
11:             return has Reentrancy;
12:           end if
13:         end if
14:       end if
15:     end for
16:   end for
17: end for

```

5.2.3 Denial of Service (DoS)

In addition to the *Denial of Service* vulnerability described in Sect. 2.2, there is also a loop call that produces a DoS situation. In the *Payment()* shown in Listing 13, there is a loop that pays the *payment* to the *lenders[i-1].addr* (line 4). However, this piece of code has a serious vulnerability, if one of the lenders is a malicious user, by manipulating *fallback()* to cause the contract to send the loan to fail, it will trigger an exception and roll back all transactions.

Listing 13 Code example: Denial of Service

```

1  function Payment() public {
2      for (uint i = lenders.length; i > 0; i--) {
3          uint payment = lenders[i-1].amount * 1000;
4          require(lenders[i-1].addr.send(payment));
5          ...
6      }
7  }

```

Requirements to be met to detect *Denial of Service*:

- (1) A non-cyclic function has a *ConCall*, which handles exceptions in the event that an external call fails (such as `if(!currentking.send(highestBid))revert();` in line 7 of Listing 2 in Sect. 2.2). Note that a failed *transfer()* automatically generates a rollback, and when the call is a high-level call (such as a user-defined call), there is no need to determine the exception handling operation because it is already integrated into the high-level call.
- (2) The contract status changes after the external call. For example, in Listing 2, the changes in the *currentking* and the *highestBid* (lines 8-9).
- (3) If there is a loop in the function, determine whether there is a transfer operation *ConCall* in the loop (such as line 4 of Listing 13).

Algorithm 6 is the process of detecting *Denial of Service*. For a function that satisfies the presence of *ConCall*, traverse the IR instructions of the nodes within the function (lines 6–8). Line 9 indicates that if the call type is *ConCall*, the destination address is not *Solidity Variable* and the destination address satisfies *is_empty* (i.e., the target address expression is null). The *Denial of Service* vulnerability is returned if the contract state is changed after the external call (lines 10-11).

Another case is that there is a loop node in the function where *ConCall* exists (line 17), and all IR instructions within the loop are traversed (line 18). If the ir is *ConCall*, the destination address type is not a *Solidity Variable* and the expression is null (line 19), then the *Denial of Service* vulnerability is output (line 20).

Algorithm 6 Detection of Denial of Service**Input:** a smart contract**Output:** there exists Denial of Service or not

```

1: funcs  $\leftarrow$  get ConCall functions list;
2: nodes  $\leftarrow$  all nodes in a function;
3: irs  $\leftarrow$  all IR instructions in a node;
4: dest  $\leftarrow$  destination address;
5: all_ir_operations  $\leftarrow$  all IR instructions within the loop;
6: for f in funcs do
7:   for n in f.nodes do
8:     for ir in n.irs do
9:       if is_instance(ir, ConCall) and not is_instance(ir.dest, SolidityVariable) and
         is_empty(ir.dest.expression) then
10:        if write_after_call(var) then
11:          return Denial of Service;
12:        end if
13:      end if
14:    end for
15:  end for
16: else
17:  if for_loop(n) then
18:    for ir in all_ir_operations do
19:      if is_instance(ir, ConCall) and not is_instance(ir.dest, SolidityVariable) and
        is_empty(ir.dest.expression) then
20:        return Denial of Service;
21:      end if
22:    end for
23:  end if
24: end for

```

5.2.4 Transaction ordering dependence (TOD)

To check the *Transaction Ordering Dependence*, the following requirements need to be met:

- (1) A function of the contract has a transfer operation, and the amount of the transfer is determined by a certain state variable.
- (2) This state variable can be modified in another function of the contract.

Algorithm 7 describes the rule representation of *Transaction Ordering Dependence* detection. First, the algorithm iterates through all the variables written in each function (*var_written*) (lines 8–10). We only need to filter the argument type of the transfer amount to be *uint* since we only consider the *Transaction Ordering Dependence* caused by the transfer operation (line 11). Add the matching variables to the *vet* set and add the name of the function in which they are located to the *fet* set (line 12). The next step is to find the function where the *ConCall* exists by iterating through the IR instructions in the nodes (lines 17–20). Determine if the transfer amount argument (*call_value*) is in the *vet* set and the function where the transfer operation is performed is not in the *fet* set (line 21), then output the *Transaction Ordering Dependence*.

Algorithm 7 Detection of Transaction Ordering Dependence**Input:** a smart contract**Output:** there exists Transaction Ordering Dependence or not

```

1: allfuncs  $\leftarrow$  get all functions list;
2: funcs  $\leftarrow$  get ConCall functions list;
3: nodes  $\leftarrow$  all nodes in a function;
4: irs  $\leftarrow$  all IR instructions in a node;
5: vet  $\leftarrow$  set of state variables written;
6: fet  $\leftarrow$  set of functions that meet the conditions;
7: call_value  $\leftarrow$  parameter of amount;
8: for f in allfuncs do
9:   for n in f.nodes do
10:    for x in n.var_written do
11:      if is_instance(x.type, uint) then
12:        vet  $\leftarrow$  x and fet  $\leftarrow$  f;
13:      end if
14:    end for
15:  end for
16: end for
17: for f in funcs do
18:   for n in f.nodes do
19:    for ir in irs do
20:      if is_instance(ir, ConCall) then
21:        if ir.call_value in vet and f not in fet then
22:          return Transaction Ordering Dependence;
23:        end if
24:      end if
25:    end for
26:  end for
27: end for

```

6 Experiments and evaluation

6.1 Experimental setup

Experimental Environment. All experiments were conducted on an Ubuntu 20.04 virtual machine with four processor cores and 4GB of memory.

Tool implementation. The ContractSentry vulnerability analysis is built on top of the Slither framework, which elevates Solidity source code to an intermediate representation.

Dataset. To evaluate ContractSentry, we obtained 16,010 real-world smart contracts from Etherscan (2023), and we divided the entire dataset into two mutually exclusive sub-datasets. The small scale dataset consists of 237 smart contracts that had been manually verified and labeled. The large scale dataset is made up of 15,773 smart contracts, which included a portion of the true positives reported by the manual verification tool.

Metrics. In the experiment, we collect three types of measurements: True Positive (TP), False Positive (FP), and False Negative (FN). TP indicates that a vulnerability in the contract is correctly detected. FP indicates erroneously reporting a vulnerability in a contract, i.e., the smart contract is actually secure but is falsely flagged

Table 3 Vulnerability detection capability of each tool

Tool	Reentrancy	DoS	TOD
Slither	✓	✓	×
Securify	✓	✓	✓
Oyente	✓	×	✓
Sailfish	✓	×	✓
Smartian	✓	×	×
ContractSentry	✓	✓	✓

as having a vulnerability. FN indicates a failure to detect a contract with a real vulnerability. Through these three measurements, we calculate three metrics to evaluate the tool: *Precision*($P\%$), *Recall*($R\%$), and *F1*($F\%$), as shown in formulas (1)-(3), respectively.

$$Precision = \frac{TP}{TP + FP} \times 100\% \quad (1)$$

$$Recall = \frac{TP}{TP + FN} \times 100\% \quad (2)$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \times 100\% \quad (3)$$

6.2 Analysis of experimental results

In this section, we evaluate the performance of our tool ContractSentry and five other state-of-the-art detection tools (i.e., Slither, Securify, Oyente, Sailfish, and Smartian) on 237 verified smart contracts, and Table 3 shows the capabilities of the six tools to detect *Reentrancy*, *Denial of Service*, and *Transaction Ordering Dependence* vulnerabilities. We summarise the detection results in Tables 4, 5, 6, 7 (where N represents the number of vulnerability types flagged in the dataset). Next, we provide an in-depth analysis of the reasons for the vulnerability results in these tables.

(1) Reentrancy. Table 4 shows the detection results of the six tools for reentrancy. In Table 5, we demonstrate in detail the detection results of each tool for four types of strategies to prevent reentrancy vulnerabilities.

Slither and Securify respectively report 62 and 68 false positives, as can be seen in Table 4. Both tools treat a state change after an external call as a reentrancy vulnerability. In addition to the four cases in Table 5 (which neither Slither nor Securify can detect), both tools also consider a state change after an external call of type is *transfer()* or *send()* as a reentrancy vulnerability. In addition, some external calls in smart contracts do not involve financial acts and the state may not be changed after the external call. For instance, Listing 14 declares a function *flushTokens()* that calls the token contract instance *balanceOf()* to get the token balance at the current contract address (line 4). Even if reentering this function, it will not affect the execution of the smart contract (e.g., the ether balance, etc.), and will not generate a reentrancy vulnerability. However, ContractSentry imposes conditional constraints on the arguments

Table 4 Experimental results of tools on Reentrancy

Tool Name	N	TP	FP	FN	P (%)	R (%)	F (%)
Slither	10	7	55	3	11.3	70	19.5
Securify	10	6	62	4	8.8	60	15.3
Oyente	10	7	10	3	41.2	70	51.9
Sailfish	10	7	6	3	53.8	70	60.8
Smartian	10	7	4	3	63.6	70	66.6
ContractSentry	10	7	2	3	77.8	70	73.7

Table 5 Analysis results of four methods to stop reentrancy vulnerabilities

Cause Type	N	Slither	Securify	Oyente	Sailfish	Smartian	ContractSentry
Identity Control	5	0	0	0	3	4	5
Reentrancy Lock	5	0	0	5	5	5	5
Fixed Address	3	0	0	0	1	3	3
No Financial Risk	8	0	0	6	6	7	8

of such calls, and experimental results show that ContractSentry greatly reduces the generation of such false positives.

The generation of false positives in Oyente is primarily due to its broad detection criteria, which can not detect *Identity Control* and *Fixed Address*. False negatives may be caused by path explosion. Sailfish does not rely on predefined rules. It starts with lightweight exploration to reduce the number of instructions that need to be analyzed. Then, it uses symbolic execution to solve constraints, thereby verifying the feasibility of counter-examples. Although this method can effectively filter external calls it does not involve financial behavior and *Reentrancy Lock*. However, the detection effect of other filter reentrancy behaviors (i.e., *Identity Control*, *Fixed Address*, and *No Financial Risk*) is not satisfactory. Although Smartian has a high precision and recall rate for reentrancy detection, it is unable to identify TOD and DoS vulnerabilities.

ContractSentry outperforms the other three tools in terms of precision and F1. The recall is better than Securify. It reports 2 false positives and 3 false negatives. The false positives are due to the lack of analysis of the fine-grained contextual content during cross-contract invocations (as is the case with the other three tools), incorrectly determining the security of certain contracts. The cause of the false negatives is the presence of state variable changes in the modifier that do not directly show up after an external call. ContractSentry ignores the existence of this type of vulnerability.

Table 6 Experimental results of tools on DoS

Tool Name	N	TP	FP	FN	P (%)	R (%)	F (%)
Slither	14	4	7	10	36.4	28.6	32
Securify	14	4	8	10	33.3	28.6	30.8
ContractSentry	14	14	3	0	82.3	100	90.3

Listing 14 An FP flagged by Slither and Securify

```

1  function flushTokens(address tokenAddr) {
2      ...
3      address forAddr = address(this);
4      uint forBalance = instance.balanceOf(forAddr);
5      if (forBalance == 0) {
6          return;
7      }
8      ...
9  }
```

(2) Denial of Service(DoS). Table 6 displays the results of Slither and Securify in identifying DoS. The results indicate a high number of false positives and false negatives. The false negatives are because both tools can only detect DoS cases in loops, ignoring cases where an external call that contains exception handling in non-loops generates DoS. The reason for the high number of false positives is that for example, in a loop call, the destination address of the external call is the contract caller. Therefore the addresses of the loop transfers are all the same, and normally there does not exist a DoS vulnerability in this case. However, these two tools mistakenly treat such smart contracts as vulnerable ones, resulting in false positives. However, ContractSentry considers both cases thoroughly, and the experimental results shown in Table 6 demonstrate that ContractSentry has excellent precision and recall (82.3% and 100%, respectively) in detecting DoS.

(3) Transaction Ordering Dependence(TOD). It can be seen from Table 7 that there are more false positives and false negatives in the test results of Securify and Oyente, which affects the precision and recall rate. Securify implements relatively lenient standards for detecting TOD vulnerabilities. For instance, there are two transfer operations in a contract that are not related to each other, which Securify considers to be a potential TOD risk. Oyente adopts a similar approach to Securify, resulting in a significant number of false positives in the detection results. Although Sailfish effectively overcomes the above problems, it still has 7 false positives and 7 false negatives.

From Table 7, we can see that ContractSentry contains 5 false positive contracts and 5 false negative contracts. The reason for false positives is that in some specific cases, the operations contained in the smart contract do not have the possibility of

Table 7 Experimental results of tools on TOD

Tool Name	N	TP	FP	FN	P (%)	R (%)	F (%)
Securify	16	9	15	7	37.5	56.3	45
Oyente	16	5	13	11	27.8	31.3	29.4
Sailfish	16	9	7	7	56.3	56.3	56.3
ContractSentry	16	11	5	5	68.8	68.8	68.8

creating TOD vulnerabilities. For instance, Listing 15 shows a real donation collection contract that transfers collected donations to the recipient. ContractSentry treats the behavior that the number of *donations* from the donation operation (line 10) can be changed in *Defray()* (line 5) as a TOD vulnerability. The fact that *recipient* will receive more donations if *Defray()* is operated ahead of *WithdrawDonations()*, but this would not result in TOD. The reason for false negatives is that we only consider the operability of the transfer amount and do not focus on the competition between function callers. Although static analysis tools have limitations in the identification of TOD vulnerabilities, ContractSentry did report contracts where transfer amounts could be manipulated, which plays an important role in ensuring the security of the contract.

Listing 15 An example of TOD false positive

```

1  contract Doac {
2    ...
3    function Defray(...) public {
4      ...
5      donations += donation;
6      ...
7    }
8    function WithdrawDonations(...) public {
9      require(msg.sender == investor);
10     recipient.transfer(donations);
11     ...
12   }
13   ...
14 }

```

6.3 Large scale dataset evaluation

In Sect. 6.2, ContractSentry showed excellent performance compared to other tools by testing on a small scale dataset. In this section, we used 15,773 smart contracts acquired from Etherscan as test samples to evaluate the performance of ContractSentry in real-

Table 8 The ability of ContractSentry in vulnerability detection

Vulnerability type	Safe	Unsafe	Proportion (%)
Reentrancy	12827	72	0.6
DoS	11528	1371	10.6
TOD	12573	326	2.5

world applications. We analyzed the detection effect of ContractSentry by analyzing the proportion of three types of vulnerabilities as shown in Table 8 and verifying some vulnerable contracts shown in Table 9.

Ability. Table 8 summarizes the ability of ContractSentry to discover vulnerabilities. We summarize two possible outcomes: *Safe* indicates no vulnerabilities detected, and *Unsafe* indicates the presence of potential vulnerabilities (note that the results displayed in Table 8 are the final results after excluding contracts that could not be analyzed). *Proportion (%)* represents the proportion of vulnerable contracts.

From Table 8, we can see that reentrancy vulnerabilities are less frequent than the other two types of vulnerabilities, accounting for only 0.6% of the total, which is mainly because most of the smart contracts adopt four strategies (i.e., *Identity Control*, *Reentrancy Lock*, *Fixed Address*, and *No Financial Risk*) to prevent reentrancy attacks. It follows that our proposed method of detecting the four methods of preventing reentrancy is necessary.

DoS vulnerabilities are relatively common, accounting for 10.6% of the total. When a change in the state of a smart contract depends on the execution of an external call, and this execution consistently fails, then it may cause the contract to fail to perform its function as expected. 326 TOD vulnerabilities were identified, accounting for 2.5% of the total. TOD vulnerabilities arise when the execution order of transactions is maliciously manipulated. It can be seen in this dataset that although the occurrence of TOD vulnerabilities is not frequent, some contracts have strict requirements for the execution order of transactions, and the wrong execution order may have adverse effects.

Reliability. To evaluate the reliability of ContractSentry, we performed a manual check to verify that the identified vulnerable contracts were indeed vulnerable. Considering the large number of vulnerabilities, we could not manually check all reported contracts. Therefore, we randomly selected 100 reported vulnerable contracts in each category for manual verification. Since the number of reentrancy vulnerabilities reported was less than 100, we examined all 72 contracts.

Table 9 shows the results of the analysis, the first column shows the type of vulnerabilities, followed by the number of true positives, the number of false positives, and the percentage of true positives of the identified vulnerabilities. As can be seen from Table 9, for the reentrancy vulnerability, 53 out of 72 reported vulnerable contracts are true positives, with a true positive rate of 73.6%.

The DoS and TOD vulnerabilities are found in 77 and 67 true positive contracts among their respective 100 sampled vulnerable contracts, with true positive rates of 77% and 67%, respectively. The verification results and the true positive rate demonstrate how effective ContractSentry is at detecting vulnerabilities.

Table 9 Manual confirmation of results

Vulnerability Type	TP	FP	P (%)
Reentrancy	53	19	73.6
DoS	77	23	77
TOD	67	33	67

7 Related work

Recently, many automated detection tools targeting vulnerabilities in smart contracts have emerged. From the perspective of software analysis, these tools can be classified into static analysis tools and dynamic analysis tools.

Among static analysis tools, Oyente is the first tool to detect smart contract vulnerabilities based on symbolic execution. It determines the execution path by constructing a control flow graph of the contract bytecode and using the Z3 solver (De Moura and Bjørner 2008). Symbolic execution has certain advantages in detecting security vulnerabilities in smart contracts, but issues such as path reachability have resulted in reduced accuracy in detection results. DefectChecker (Chen et al. 2021) is another testing tool that uses symbolic execution. Unlike Oyente, DefectChecker uses stack events and feature detection instead of the SMT solver, which improves detection efficiency to a certain extent. Both Securify and Vandal (Brent et al. 2018) discover vulnerabilities by transforming contract bytecode into an intermediate representation. They express the extracted semantic information into semantic facts through Datalog language (Maier et al. 2018) and conduct vulnerability detection according to predefined rules. Slither and Smartcheck (Tikhomirov et al. 2018) transform the Solidity source code into an intermediate representation and generate vulnerability detection patterns. Slither creates an intermediate representation called SlithIR to express the contract source code. Smartcheck transforms the smart contract Solidity source code into an XML-based intermediate representation and then uses XPath (2023e) matching to detect vulnerabilities in smart contracts.

These static tools mentioned above are designed to detect vulnerabilities by setting up different patterns, but the design of these pattern rules is incomplete, resulting in accurate analysis of complex contracts (such as the strategies used in contracts to prevent vulnerabilities). However, ContractSentry analyzes the context information within functions of potentially vulnerable contracts and reports benign vulnerabilities to exclude false alarms. In this paper, we used Oyente, Slither, Securify, Sailfish, and Smartian as baseline methods to demonstrate that ContractSentry is more accurate.

There are other static analysis tools for vulnerability detection, Zeus (Kalra et al. 2018) uses abstract interpretation and symbolic model methods for vulnerability detection of smart contracts. By transforming the source code into LLVM intermediate language and then using the SeaHorn (Gurfinkel et al. 2015) verification framework for formal verification. Formal verification tools can effectively eliminate ambiguity and non-generality in contracts but mostly lack a high degree of automation.

In addition, there are some dynamic analysis tools. ContractFuzzer is the first fuzz testing tool for smart contracts on Ethereum. It takes randomly generated transactions

as test inputs, uses EVM to record contract operation behaviors, and performs security analysis. sFuzz (Nguyen et al. 2020) draws on the idea of the traditional fuzz testing tool AFL and introduces a feedback-guided fuzzing engine to improve the coverage of test cases. Although dynamic detection tools can produce fewer false positives, the code coverage is not ideal because the test cases are random. Moreover, these tools are not suitable for large-scale detection due to lower efficiency. In contrast, ContractSentry as a static analysis tool enables a high degree of code coverage. And it does not need to execute contracts, so it has excellent detection efficiency.

8 Conclusion

In this paper, we propose ContractSentry, a smart contract vulnerability detection tool at the Solidity source code level. To solve the problem of the high rate of false negatives and false positives in previous vulnerability detection tools, ContractSentry first constructs contract information from the Solidity code level and transforms it into an intermediate representation. Finally, by extracting the critical information and using the intermediate representations to construct comprehensive vulnerability detection rules. We evaluated ContractSentry using datasets obtained from Etherscan. The results show that compared with the existing state-of-the-art vulnerability detection tools, ContractSentry enhances the accuracy of detection.

ContractSentry may have false positives or false negatives for some smart contracts that involve cross-contract invocations. This is due to the limitations of the static analysis method, which cannot simulate the dynamic execution of the contract, and may not find some vulnerabilities that depend on runtime state or input. In the future, we plan to explore the possibility of combining ContractSentry with dynamic detection methods to detect cross-contract vulnerabilities, improving the comprehensiveness and accuracy of vulnerability detection.

Author Contributions All authors contributed to the conception and design of the study. Shiji Wang wrote the manuscript. All authors read and approved the final manuscript.

Funding This work was supported by the National Natural Science Foundation of China (Grant Nos. 61972360 and 62072392), and Shandong Provincial Natural Science Foundation (Grant No. ZR2024MF111).

Data availability The datasets generated and analysed during the current study are available from the corresponding author on reasonable request.

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose.

Ethical approval/informed consent The data used in this study was exclusively generated by the authors. No third parties were involved in the data generation. No research involving human participants or animals has been performed.

References

- Alqarni, M.A., Alkatheiri, M.S., Chauhdary, S.H., Saleem, S.: Use of Blockchain-Based Smart Contracts in Logistics and Supply Chains. *MDPI* (2023)
- Boroń, M., Kobusińska, A.: Alternative authentication with smart contracts for online games. In: 2021 IEEE 46th Conference on Local Computer Networks (LCN), pp. 415–418 (2021). IEEE
- Bose, P., Das, D., Chen, Y., Feng, Y., Kruegel, C., Vigna, G.: Sailfish: Vetting smart contract state-inconsistency bugs in seconds. In: 2022 IEEE Symposium on Security and Privacy (S&P), pp. 161–178 (2022). IEEE
- Brent, L., Jurisevic, A., Kong, M., Liu, E., Gauthier, F., Gramoli, V., Holz, R., Scholz, B.: Vandal: A scalable security analysis framework for smart contracts. *arXiv preprint arXiv:1809.03981* (2018)
- Chen, J., Xia, X., Lo, D., Grundy, J., Luo, X., Chen, T.: Defining smart contract defects on ethereum. *IEEE Transactions on Software Engineering* **48**(1), 327–345 (2020)
- Chen, J., Xia, X., Lo, D., Grundy, J., Luo, X., Chen, T.: Defectchecker: Automated smart contract defect detection by analyzing evm bytecode. *IEEE Transactions on Software Engineering* **48**(7), 2189–2207 (2021)
- Chen, Q., Zhou, T., Liu, K., Li, L., Ge, C., Liu, Z., Klein, J., Bissyandé, T.F.: Tips: towards automating patch suggestion for vulnerable smart contracts. *Automated Software Engineering* **30**(2), 31 (2023)
- Choi, J., Kim, D., Kim, S., Grieco, G., Groce, A., Cha, S.K.: Smartian: Enhancing smart contract fuzzing with static and dynamic data-flow analyses. In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 227–239 (2021). IEEE
- De Moura, L., Björner, N.: Z3: An efficient smt solver. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems, pp. 337–340 (2008). Springer
- Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P.: Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE), pp. 530–541 (2020)
- Etherscan. <https://etherscan.io/> Accessed on 22 June 2023
- Feist, J., Grieco, G., Groce, A.: Slither: a static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), pp. 8–15 (2019). IEEE
- Guo, H., Yu, X.: A survey on blockchain technology and its security. *Blockchain: research and applications* **3**(2), 100067 (2022)
- Gurfinkel, A., Kahsai, T., Komuravelli, A., Navas, J.A.: The seahorn verification framework. In: International Conference on Computer Aided Verification, pp. 343–361 (2015). Springer
- Jiang, B., Liu, Y., Chan, W.K.: Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, pp. 259–269 (2018)
- John, K., Kogan, L., Saleh, F.: Smart contracts and decentralized finance. *Annual Review of Financial Economics* **15**, 523–542 (2023)
- Kalra, S., Goel, S., Dhawan, M., Sharma, S.: Zeus: analyzing safety of smart contracts. In: *Ndss*, pp. 1–12 (2018)
- Khan, Z.A., Namin, A.S.: Ethereum smart contracts: Vulnerabilities and their classifications. In: 2020 IEEE International Conference on Big Data (Big Data), pp. 1–10 (2020). IEEE
- Luu, L., Chu, D.-H., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 254–269 (2016)
- Maier, D., Tekle, K.T., Kifer, M., Warren, D.S.: Datalog: concepts, history, and outlook. In: *Declarative Logic Programming: Theory, Systems, and Applications*, pp. 3–100 (2018)
- Nerurkar, P., Bhirud, S., Patel, D., Ludinard, R., Busnel, Y., Kumari, S.: Supervised learning model for identifying illegal activities in bitcoin. *Applied Intelligence* **51**, 3824–3843 (2021)
- Nguyen, T.D., Pham, L.H., Sun, J., Lin, Y., Minh, Q.T.: sfuzz: An efficient adaptive fuzzer for solidity smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 778–788 (2020)
- Qian, P., Liu, Z., He, Q., Huang, B., Tian, D., Wang, X.: Smart contract vulnerability detection technique: A survey. *arXiv preprint arXiv:2209.05872* (2022)
- SlithIR Types. <https://github.com/crytic/slither/wiki/SlithIR/> Accessed on 22 June 2023
- Slowmist: Slowmist. <https://hacked.slowmist.io/en/> Accessed on 12 November 2023
- Solidity. <http://solidity.readthedocs.org/> Accessed on 26 June 2023

- The Multi-sig Hack: A Postmortem. <https://www.parity.io/the-multi-sig-hack-a-postmortem/> Accessed on 26 June 2023
- Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., Alexandrov, Y.: Smartcheck: Static analysis of ethereum smart contracts. In: Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain, pp. 9–16 (2018)
- Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Buenzli, F., Vechev, M.: Securify: Practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 67–82 (2018)
- Understanding the DAO Attack. <https://www.coindesk.com/understanding-dao-hack-journalists/> Accessed on 26 June 2023
- Wang, W., Huang, H., Yin, Z., Gadekallu, T.R., Alazab, M., Su, C.: Smart contract token-based privacy-preserving access control system for industrial internet of things. *Digital Communications and Networks* **9**(2), 337–346 (2023)
- Wood, G., et al.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper **151**(2014), 1–32 (2014)
- XML Path Language. <https://www.w3.org/TR/xpath-3/> Accessed on 22 June 2023
- Zheng, Z., Zhang, N., Su, J., Zhong, Z., Ye, M., Chen, J.: Turn the rudder: A beacon of reentrancy detection for smart contracts on ethereum, p. 2303 (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.